

**TALLER CAMPUSTASKS - ENTORNO, ACTUALIZACIÓN
Y ELIMINACIÓN DE TAREAS**



PRESENTADO A:

KEVIN STEVEN VICTORIA OSPINA

PRESENTADO POR:

PAULA JIMENA BOHORQUEZ BERMÚDEZ - 2459409

MANUELA DELGADO AGUIRRE - 2459640

JUAN CAMILO GIL AGUDELO - 2459531

**UNIVERSIDAD DEL VALLE
FACULTAD DE INGENIERÍA
DESARROLLO DE SOFTWARE II
TULUÁ, VALLE
2026**

NOMBRE DE LA RAMA UTILIZADA:

- taller/actualizar-eliminar-tareas-bohorquez-delgado-gil

Puesta en marcha

Entorno: Fedora 44, [Node.js](#) 24.x, PostgreSQL 18

1. Instalación de PostgreSQL

Se instaló PostgreSQL 18 desde el repositorio oficial (PGDG), siguiendo la guía de postgresql.org para fedora.

```
sudo dnf install -y
https://download.postgresql.org/pub/repos/yum/reporpms/F-44-x86_64/pgdg-
fedora-repo-latest.noarch.rpm
sudo dnf install -y postgresql18-server
sudo /usr/pgsql-18/bin/postgresql-18-setup initdb
sudo systemctl enable postgresql-18
sudo systemctl start postgresql-18
```

Después de instalado se creo un usuario con contraseña para que el backend pudiese conectarse al localhost (127.0.0.1):

```
sudo -u postgres createuser --superuser camilog03
psql -d postgres -c "ALTER USER camilog03 PASSWORD '*****';"
```

Comprobación:

```
sudo -u postgres createuser --superuser camilog03 psql -d postgres -c
"ALTER USER camilog03 PASSWORD '*****';"
```

Con este comando se confirmó que la consola de psql estaba conectada, como se

```
[fedora] as camilog03 in ~
└─ psql -U camilog03 -h 127.0.0.1 -p 5432 -d postgres
Password for user camilog03:
psql (18.6)
Type "help" for help.

postgres=#
```

2. Base de datos, tabla y datos iniciales

Cómo el repositorio no trae ningún script SQL, la base de datos se tuvo que crear a pie desde la consola de 'psql'

```
CREATE DATABASE campus_tasks;
\c campus_tasks
CREATE TABLE tareas (id SERIAL PRIMARY KEY, titulo TEXT NOT NULL);
INSERT INTO tareas (titulo) VALUES
('Leer la guía de la clase 2'),
('Preparar el entorno de desarrollo');
```

Para revisar que toda la operación fue exitosa se realizó el siguiente comando

```
SELECT * FROM tareas;
```

Siendo su salida la siguiente

```
campus_tasks=# SELECT * FROM tareas;
 id |          titulo
----+-----
  1 | Leer la guía de la clase 2
  2 | Preparar el entorno de desarrollo
(2 rows)
```

3. Archivo 'backend/.env'

Era necesario crear un archivo, en este caso el .env quien almacena las cinco variables de entorno que se necesitan para generar la conexión con la base de datos anteriormente creada y el proyecto, el archivo quedó de la siguiente manera

```
l: nvim .env
3 DB_HOST=127.0.0.1
2 DB_PORT=5432
1 DB_USER=camilog03
4 DB_PASSWORD=*****|
1 DB_NAME=campus_tasks
```

4. Backend

Inicio del Backend

```
cd backend
npm ci
npm run start:dev
```

```
(/*Д*/ npm start

> backend@0.0.1 start
> nest start

[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [NestFactory] Starting Nest application...
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [InstanceLoader] AppModule dependencies initialized +14ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [InstanceLoader] DatabaseModule dependencies initialized +0ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [InstanceLoader] TareasModule dependencies initialized +2ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [RoutesResolver] TareasController {/tareas}: +4ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [RouterExplorer] Mapped {/tareas, GET} route +3ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [RouterExplorer] Mapped {/tareas, POST} route +1ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [RouterExplorer] Mapped {/tareas/:id, PATCH} route +4ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [RouterExplorer] Mapped {/tareas/:id, DELETE} route +1ms
[Nest] 316764 - 10/05/2026, 11:29:03 PM LOG [NestApplication] Nest application successfully started +2ms
```

JSON Datos sin procesar Cabeceras

Guardar Copiar Contraer todo Expandir todo 🔍 Filtrar JSON

▼ 0:

id: 1

titulo: "Leer la guía de la clase 2"

▼ 1:

id: 2

titulo: "Preparar el entorno de desarrollo"

5. Frontend

Inicio del Frontend

```
cd frontend
npm ci
npm start
```

Resultados de '<http://localhost:4200>', aparecen las dos tareas, y al escribir una nueva tarea y pulsar **Agregar** la tarea nueva se mostrará en la lista.



6. Pruebas existentes

Backend (npm test desde backend): pasan las dos suites, tareas.service.spec.ts y tareas.integration.spec.ts.)

Frontend (npm test desde frontend): fue necesario instalar Google Chrome, porque Karma lo usa para ejecutar las pruebas. Pasan las pruebas de tareas.component.spec.ts.

7. Lectura de los ejemplos

Las tres pruebas de ejemplo siguen el mismo patrón de tres pasos, preparar, ejecutar, verificar, y ninguna abre PostgreSQL ni necesita el backend encendido: la dependencia externa siempre se reemplaza por un objeto simulado.

- **backend/src/tareas/tareas.service.spec.ts (unitaria, Jest):** Crea un módulo de prueba con TareasService real y un DatabaseService falso cuyo único método, query, es un jest.fn(). Cada prueba define qué devuelve query (mockResolvedValue({ rows: [...] })), llama directamente al método del servicio y verifica dos cosas: el valor devuelto y que query recibió exactamente el SQL y los parámetros esperados. En beforeEach se hace query.mockReset() para que una prueba no contamine a la siguiente.
- **backend/src/tareas/tareas.integration.spec.ts (integración, Jest + supertest):** Arma un TestingModule con el controlador y el servicio reales y el mismo DatabaseService simulado, y levanta una aplicación Nest en memoria. Con supertest se hace una petición HTTP real (método, ruta y cuerpo) y se verifica el código de estado, el cuerpo de la respuesta y los argumentos de query. A diferencia de la unitaria, aquí también se prueba la ruta, la lectura del cuerpo y el código HTTP.
- **frontend/src/components/tareas/tareas.component.spec.ts (componente, Jasmine + Karma):** Reemplaza el servicio HTTP por un spy (jasmine.createSpyObj) que devuelve observables con of(...). Renderiza TareasComponent, simula lo que hace el usuario (escribir en el campo y hacer clic en Agregar) y verifica que el spy se llamó con los argumentos correctos y que la pantalla muestra el resultado.

Patrón reutilizado. Las pruebas nuevas se agregaron a los mismos archivos y con la misma estructura: en el backend, query simulado con las filas que debe devolver la base (o rows: [] para el caso 404) y verificación del SQL exacto con sus parámetros. No se usó backend/test/app.e2e-spec.ts como modelo, porque levanta la aplicación completa e intenta usar PostgreSQL de verdad.

8. Implementación

- **Backend:**

- Tareas.service.ts :

Método	SQL	Parámetros
actualizar(id, titulo)	UPDATE tareas SET titulo = \$1 WHERE id = \$2 RETURNING id, titulo	[titulo, id]
eliminar(id)	DELETE FROM tareas WHERE id = \$1 RETURNING id, titulo	[id]

- Tareas.controller.ts :

Ruta	Cuerpo	Si existe	Si no existe
PATCH /tareas/:id	{ "titulo": "..." }	200 y la tarea actualizada	404
DELETE /tareas/:id	ninguno	200 y la tarea eliminada	404

- Decisión 1: cómo se convierte el :id a número. El :id llega como texto en la URL. Se usa el pipe de NestJS ParseIntPipe en el parámetro (@Param('id', ParseIntPipe) id: number), que lo convierte a number antes de que entre al método. Se eligió en lugar de Number(id) o parseInt porque además valida: si el valor no es un entero (por ejemplo /tareas/abc), Nest responde 400 por sí solo y la consulta nunca llega a la base.
- Decisión 2: cómo se detecta que la tarea no existe. El controlador revisa lo que devuelve el servicio: si es undefined (la sentencia no afectó ninguna fila), lanza NotFoundException, que Nest traduce a una respuesta 404 con el mensaje No existe la tarea con id <id>. Así se resuelve con una sola consulta, sin hacer primero un SELECT para comprobar la existencia, y la decisión del código de estado queda en el controlador, que es su responsabilidad; el servicio solo conoce el SQL.

- **Frontend**

-Servicio HTTP (Tareas.service.ts) :

Método	Petición
actualizar(id, titulo)	PATCH http://localhost:3000/tareas/:id con { titulo }
eliminar(id)	DELETE http://localhost:3000/tareas/:id

Ambos devuelven un Observable<Tarea> con la tarea que responde el backend, igual que crear.

-Componente (TareasComponent) :

Cada tarea de la lista ofrece dos acciones, con los textos exactos Editar, Guardar y Eliminar:

- Editar. Se agregó la señal estadoEdicion, que guarda el id de la tarea que se está editando (o null). Al pulsar Editar, esa fila cambia el título por un campo de texto con el valor actual y un botón Guardar; las demás filas no cambian. Al pulsar Guardar se llama a actualizar(id, titulo) y, cuando el backend responde, se reemplaza en la lista solo esa tarea por la que devolvió el backend y se sale del modo edición.
- Eliminar. Llama a eliminar(id) y, cuando el backend responde, se quita de la lista la tarea con ese id. Las demás siguen visibles.

Qué pasa ante un error. La lista solo se modifica dentro de la respuesta del servicio (subscribe), nunca antes de llamarlo. Para eso subscribe recibe dos funciones: next, que se ejecuta cuando el backend responde con éxito, y error, que se ejecuta cuando responde con un error.

Decisión: qué mostrar si el backend responde 404. Un 404 significa que la tarea ya no existe en la base (por ejemplo, porque se eliminó desde otra pestaña), así que la pantalla estaba desactualizada. En ese caso se quita esa tarea de la lista, tanto al eliminar como al guardar, y al guardar además se sale del modo edición. Así la lista vuelve a coincidir con lo que hay en el backend, que es el mismo resultado que el usuario vería al recargar la página. Ante cualquier otro error (por ejemplo, el backend apagado) la lista no cambia, porque no se sabe si la operación se hizo.

9. Pruebas Nuevas

- Backend

Tipo	Archivo	Caso	Qué verifica
Unitaria	tareas.service.spec.ts	actualizar	query recibe el UPDATE con [titulo, id]; el método devuelve la fila actualizada
Unitaria	tareas.service.spec.ts	eliminar	query recibe el DELETE con [id]; el método devuelve la fila eliminada
Integración	tareas.integration.spec.ts	PATCH existente	responde 200 con la tarea actualizada; argumentos de query correctos
Integración	tareas.integration.spec.ts	PATCH inexistente	con rows vacío responde 404
Integración	tareas.integration.spec.ts	DELETE existente	responde 200 con la tarea eliminada; argumentos de query correctos
Integración	tareas.integration.spec.ts	DELETE inexistente	con rows vacío responde 404

Resultado:

```
[fedora] as camilog03 in ~/Universidad/6_Semestre/Desar
(/"A")/ npm test

> backend@0.0.1 test
> jest

PASS src/tareas/tareas.integration.spec.ts
PASS src/tareas/tareas.service.spec.ts

Test Suites: 2 passed, 2 total
Tests:      10 passed, 10 total
Snapshots:  0 total
Time:       1.031 s
Ran all test suites.
```

- **Frontend**

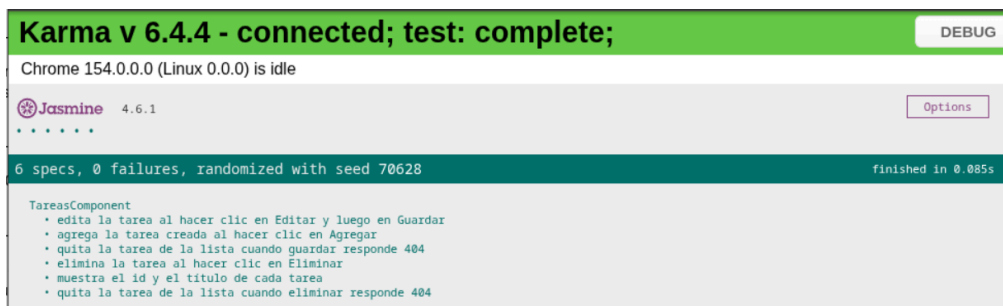
Todas están en `frontend/src/components/tareas/tareas.component.spec.ts`. El *spy* del servicio se amplió con los métodos actualizar y eliminar, y se agregó una función auxiliar `boton(fila, texto)` que busca un botón dentro de una fila por su texto exacto (Editar, Guardar, Eliminar). Las pruebas nuevas parten de una lista de dos tareas, para poder comprobar que la otra tarea no se ve afectada.

Caso	Qué verifica
Eliminar una tarea	Tras pulsar Eliminar, el <i>spy</i> eliminar se llamó con el id de esa tarea; la tarea ya no aparece y la otra sigue visible
Editar una tarea	Tras pulsar Editar, escribir un título y pulsar Guardar, el <i>spy</i> actualizar se llamó con el id y el título escrito; la pantalla muestra el título que devolvió el backend y la otra tarea no cambia
Eliminar responde 404	Con el <i>spy</i> devolviendo un <code>HttpErrorResponse 404</code> , la tarea se quita de la lista y la otra sigue visible
Guardar responde 404	Con el <i>spy</i> devolviendo un <code>404</code> , la tarea se quita de la lista, desaparece el campo de edición y <code>estadoEdicion</code> vuelve a <code>null</code>

En la prueba de editar, el *spy* devuelve un título distinto al que escribió el usuario (Título devuelto por backend). Eso comprueba que la pantalla muestra lo que respondió el backend y no lo que se escribió en el campo.

Las dos pruebas anteriores (listar y agregar) no se modificaron en lo que verifican. En total el archivo tiene 6 pruebas.

Resultado:



```
Karma v 6.4.4 - connected; test: complete;
Chrome 154.0.0.0 (Linux 0.0.0) is idle
Jasmine 4.6.1
6 specs, 0 failures, randomized with seed 70628 finished in 0.085s

TareasComponent
  • edita la tarea al hacer clic en Editar y luego en Guardar
  • agrega la tarea creada al hacer clic en Agregar
  • quita la tarea de la lista cuando guardar responde 404
  • elimina la tarea al hacer clic en Eliminar
  • muestra el id y el título de cada tarea
  • quita la tarea de la lista cuando eliminar responde 404
```

10. GIT

Se trabajó en una rama nueva creada desde master, sin hacer commits en la rama principal:

- `git checkout master`
- `git pull`
- `git checkout -b taller/actualizar-eliminar-tareas-bohorquez-delgado-gil`
- `git push -u origin taller/actualizar-eliminar-tareas-bohorquez-delgado-gil`

Las demás integrantes la trajeron con:

- `git fetch`
- `git checkout taller/actualizar-eliminar-tareas-bohorquez-delgado-gil`

Antes de cada commit se ejecutó `git status` para confirmar que `.env` no aparecía, y después `git pull` y `git push`.

Commits de la rama:

Commit	Mensaje
9cf3734	Agrega el método actualizar al servicio de tareas con su prueba unitaria
c851c4c	Agrega la ruta PATCH /tareas/:id con respuesta 404 y sus pruebas de integración
80d0875	Agrega el método eliminar al servicio de tareas con su prueba unitaria
191c6ca	Agrega la ruta DELETE /tareas/:id con respuesta 404 y sus pruebas de integración
b606a27	Frontend: agregar metodos actualizar y eliminar al servicio HTTP de tareas
8ad5ae2	Frontend: agregar botones Editar, Guardar y Eliminar en la lista de tareas

dc4e9bd	Frontend: agregar pruebas de componente para editar y eliminar tareas
3dd9a4e	Frontend: quitar de la lista la tarea que el backend responde como inexistente y agregar pruebas del manejo de error 404 al editar y eliminar

URL de la rama:

<https://github.com/TevenV27/campus-task/tree/taller/actualizar-eliminar-tareas-bohorquez-de-lgado-gil>